SH

StudentHub

Guide complet du projet

Conception, architecture Flutter, gestion des utilisateurs et des réclamations avec SQLite locale.

À qui s'adresse ce guide ?
Aux personnes qui découvrent StudentHub, Flutter ou la base de données du projet et souhaitent comprendre les fonctionnalités, les dossiers du code, le fonctionnement des données et le lancement Android.

DOCUMENT PÉDAGOGIQUE · OCTOBRE 2026

Le projet en bref	Choix retenu
Application	StudentHub, application Flutter de gestion des utilisateurs et des réclamations liées au logement.
Stockage	SQLite local via sqflite. Les données restent sur l'appareil.
Plateformes visées	Android, iOS et macOS dans la configuration actuelle.
Modules présents	Inscription, connexion, profil, accueil et réclamations.

En une phrase
StudentHub est un prototype Flutter hors ligne : il permet de créer un compte local, gérer un profil et enregistrer le suivi de réclamations sur le même appareil.

Sommaire

01 Présentation, objectifs et périmètre
02 Utilisateurs, rôles et besoins
03 Parcours dans l'application
04 Architecture et organisation du code
05 Modèles de données
06 Schéma de la base SQLite
07 Comment le code lit et écrit les données
08 Authentification et profil
09 Gestion des réclamations
10 Photos et fichiers locaux
11 État Flutter avec Provider
12 Lancer le projet sur Android
13 Comprendre les erreurs de compilation rencontrées
14 Sécurité, limites et sauvegarde
15 Améliorations et feuille de route
16 Exercices et questions de soutenance
17 Glossaire et fichiers importants

Lecture conseillée
Lis d'abord les chapitres 1 à 6 pour comprendre le produit et ses données. Les chapitres 7 à 11 décrivent le fonctionnement interne. Les chapitres 12 à 16 sont un guide pratique pour lancer, expliquer et améliorer StudentHub.

01 · Présentation, objectifs et périmètre

StudentHub est une application mobile orientée logement étudiant. Le projet regroupe plusieurs idées de services, mais la version présente se concentre sur deux modules fonctionnels : la gestion des utilisateurs et la gestion des réclamations. Elle propose un parcours qui va de l'inscription au suivi local d'une demande.

Objectifs du projet

- Construire une interface mobile Flutter claire et navigable.
- Séparer les écrans, les règles métier et le stockage.
- Enregistrer des comptes et des réclamations dans une base relationnelle locale.
- Conserver une session locale entre deux ouvertures de l'application.
- Afficher les états de chargement et les erreurs au bon moment.
- Savoir expliquer les limites du prototype et les évolutions nécessaires.

Produit local ou plateforme en ligne ?

Dans ce projet, « utilisateur » signifie un compte enregistré dans la base SQLite de l'appareil. Un téléphone A et un téléphone B ont chacun leur propre fichier de base. Ils ne voient pas spontanément les mêmes comptes ou les mêmes réclamations. Le rôle propriétaire peut afficher davantage de lignes dans la base locale, mais cela ne rend pas les données visibles sur un autre appareil.

À retenir
SQLite convient à un prototype mobile autonome. Une plateforme multi-utilisateur nécessite une API et une base centrale, avec authentification et autorisations côté serveur.

Périmètre fonctionnel actuel

- Inscription et connexion par adresse e-mail ou téléphone.
- Profil : consultation, modification, photo locale et préférence de notification.
- Réclamations : création, liste, détail, modification, statut et suppression.
- Historique des changements de statut.
- Suppression d'un compte et des données qui lui sont liées.

Hors périmètre de cette version

- Catalogue et contrôle des locations actives.
- Réinitialisation réelle du mot de passe par e-mail.
- Synchronisation entre appareils et notifications poussées.
- Messagerie, paiement, réservation ou administration centralisée.

02 · Utilisateurs, rôles et besoins

L'inscription permet de choisir un rôle principal. Le modèle prévoit les valeurs locataire, propriétaire et prestataire. Le profil peut aussi conserver un rôle secondaire facultatif. Ces valeurs décrivent un type de compte, mais elles ne constituent pas à elles seules une sécurité serveur.

Rôle	Utilité dans le projet	Limite actuelle
Locataire / étudiant	Créer et gérer son profil, déposer puis suivre ses réclamations.	La demande n'est pas contrôlée contre une location officielle.
Propriétaire	Consulter la liste globale locale et changer le statut d'une demande selon l'écran.	La liste globale ne comprend que la base de cet appareil.
Prestataire	Créer un compte avec le rôle prestataire.	Le module d'affectation et de traitement d'interventions n'est pas complet.

Besoins métier principaux

- Empêcher deux comptes locaux d'utiliser la même adresse e-mail ou le même téléphone.
- Retrouver une session après fermeture de l'application.
- Rattacher une réclamation au compte qui l'a créée.
- Suivre une demande dans le temps, pas seulement conserver son état actuel.
- Conserver les photos sans stocker leurs octets directement dans une colonne SQL.

Référence de logement
Le formulaire demande une référence libre, conservée dans locationId. Il n'existe pas de table locations dans ce schéma; la valeur n'est donc pas une clé étrangère et ne vérifie pas qu'un contrat est actif.

03 · Parcours dans l’application

Étape	Action	Ce que fait l’application
1	Ouvrir StudentHub	Les providers sont créés et l’identifiant de session est lu dans SQLite.
2	Pas de session	LoginScreen s’affiche; RegisterScreen peut être ouvert.
3	S’inscrire	Le formulaire valide les champs; le service crée le compte et sa session locale.
4	Entrer dans l’app	DashboardScreen montre Accueil, Réclamations et Profil.
5	Déposer une demande	Le formulaire valide les données et copies des photos facultatives.
6	Suivre la demande	La liste et la fiche affichent statut, détails et historique.
7	Se déconnecter	La clé de session est supprimée; l’écran de connexion revient.

Flux simplifié

De l’interface vers le stockage et retour

```
main.dart
  └─ MultiProvider
      ├─ AuthProvider ── AuthService ── AppDatabase
      ├─ UserProvider ── UserService ── AppDatabase
      └─ ComplaintProvider ── ComplaintService ── AppDatabase

Écran → Provider → Service → SQLite
SQLite → Service → Provider → écran
```

04 · Architecture et organisation du code

L’application est découpée en couches simples. Chaque dossier possède une responsabilité, ce qui limite le mélange entre les détails visuels et les requêtes de stockage.

Dossier	Responsabilité	Fichiers représentatifs
lib/screens/	Écrans et interactions de l’utilisateur.	auth/, home/, profile/, complaints/
lib/providers/	État observable : données, chargement, erreur.	AuthProvider, UserProvider, ComplaintProvider
lib/services/	Accès à SQLite et logique de persistance.	AppDatabase, AuthService, UserService, ComplaintService
lib/models/	Objets métier typés.	UserModel, ComplaintModel
lib/core/	Thème, constantes, validation, routes.	AppTheme, AppConstants, Validators, AppRoutes
android/	Hôte natif et configuration Gradle Android.	Manifest, MainActivity, wrapper Gradle
test/	Tests Flutter.	widget_test.dart

Rôle des couches

- Screen : collecte la saisie, appelle une action et dessine le résultat.
- Provider : expose les données courantes et notifie les widgets avec ChangeNotifier.
- Service : exécute les opérations de base de données et convertit modèles et lignes SQLite.
- Model : décrit les données sans porter la logique d’affichage ou les requêtes SQL.

Pour suivre une action dans le code, cherche l’appel context.read<...Provider>() depuis le bouton, ouvre la méthode du provider, puis la méthode du service appelée.

05 · Modèles de données

Les modèles sont des objets Dart typés. Une ligne SQLite est une Map; les services font la conversion entre la Map stockée et l'objet affiché.

UserModel

Champ	Type	Rôle
id	String	Identifiant du compte.
nom, prenom	String	Identité affichée.
email, telephone	String	Coordonnées d'identification.
rolePrincipal	UserRole	locataire, proprietaire ou prestataire.
roleSecondaire	UserRole?	Rôle facultatif.
photo	String?	Chemin local du portrait.
preferences	Map	Préférences transformées en JSON pour SQLite.

ComplaintModel

Champ	Type	Rôle
id, userId	String	Identifiant de la réclamation et de son créateur.
locationId	String	Référence de logement saisie librement.
subject, description	String	Résumé et détails du problème.
category	ComplaintCategory	bruit, panne, loyer, voisinage ou autre.
photos	List	Chemins locaux encodés en JSON dans SQLite.
status	ComplaintStatus	ouverte, enCours ou cloturee.
createdAt	DateTime	Date convertie en texte ISO-8601.
history	List	Dates et statuts successifs.

Pourquoi des enum ?

Une enum limite une valeur à un ensemble connu. Au stockage, le service écrit le nom textuel; à la lecture, il retrouve l'enum. Si une ancienne version de la base contient une valeur inconnue, le service utilise une valeur de repli. Si l'on renomme une valeur enum, une migration doit préserver les lignes déjà enregistrées.

06 · Schéma de la base SQLite

La base s'appelle studenthub.db et se trouve dans le répertoire de base de données géré par sqflite. AppDatabase.instance ouvre la base une seule fois par processus et crée les tables lorsque le fichier est initialisé. La version du schéma est actuellement 1.

Table	Clés	Données
users	id PRIMARY KEY	Profil, e-mail et téléphone uniques, rôles, préférences, photo, hash et sel.
app_settings	key PRIMARY KEY	Petits paramètres locaux comme currentUserId.
complaints	id PRIMARY KEY; userId → users.id	Référence, objet, catégorie, description, photos, statut et date.

Table	Clés	Données
complaint_history	id INTEGER PRIMARY KEY; complaintId → complaints.id	Chaque changement de statut et sa date.

Relations

Relations entre tables

```
users (1) ──────────── (N) complaints (1) ──────────── (N) complaint_history
   id  <──────────────── userId       id  <──────────────── complaintId

app_settings : key → value (ex. currentUserId)
```

Règles de qualité des données

- NOT NULL empêche l'absence de champs obligatoires.
- UNIQUE empêche la réutilisation d'un e-mail ou d'un téléphone.
- COLLATE NOCASE rend la comparaison d'e-mail insensible aux majuscules.
- Les clés étrangères relient compte, réclamation et historique.
- ON DELETE CASCADE supprime les lignes enfants quand le parent est effacé.
- L'index complaints_user_date accélère la liste d'un utilisateur par date.
- PRAGMA foreign_keys = ON active la vérification des liens dans SQLite.

> **Types SQLite**
> SQLite utilise les classes de stockage NULL, INTEGER, REAL, TEXT et BLOB. Ici les enum et dates sont en TEXT; les photos sont des chemins dans une chaîne JSON, pas des BLOB.

07 · Comment le code lit et écrit les données

L'ouverture récupère le chemin système de la base, le combine avec le nom studenthub.db et appelle openDatabase avec une version. La fonction onConfigure active les clés étrangères; onCreate construit les tables.

Ouverture simplifiée

```
final root = await getDatabasesPath();
final db = await openDatabase(
  p.join(root, 'studenthub.db'),
  version: 1,
  onConfigure: (db) async =>
      db.execute('PRAGMA foreign_keys = ON'),
  onCreate: (db, version) async {
    await db.execute('CREATE TABLE ...');
  },
);
```

CRUD : créer, lire, modifier, supprimer

Opérations de base

```
await db.insert('users', row);

final rows = await db.query(
  'users', where: 'id = ?', whereArgs: [userId], limit: 1,
);

await db.update('users', values,
    where: 'id = ?', whereArgs: [userId]);

await db.delete('users',
    where: 'id = ?', whereArgs: [userId]);
```

Requêtes paramétrées

Les valeurs fournies par l'utilisateur passent dans whereArgs et non par concaténation de texte SQL. Cette pratique évite les problèmes de guillemets et réduit le risque d'injection SQL. Les noms des tables et colonnes doivent rester déterminés par le code.

Transactions

Créer une réclamation touche deux tables : complaints et complaint_history. Le service effectue ces écritures ensemble dans une transaction. Soit toutes réussissent, soit SQLite les annule; la demande ne reste pas sans statut initial.

Exemple de transaction

```
await db.transaction((txn) async {
  await txn.insert('complaints', complaintRow);
  await txn.insert('complaint_history', initialStatusRow);
});
```

08 · Authentification et profil

Inscription

RegisterScreen contrôle les champs requis, l'e-mail, la longueur minimale du mot de passe et l'acceptation des conditions. Le nom complet est séparé en prénom et nom. AuthProvider relaie la demande à AuthService, qui calcule le hash et écrit le compte et la session dans une transaction.

Mot de passe

AuthService utilise PBKDF2 avec HMAC-SHA256, 120 000 itérations et un sel aléatoire de 16 octets par compte. Le hash et le sel sont encodés en Base64; le mot de passe brut n'est pas conservé dans la table. À la connexion, le hash est recalculé et comparé en temps constant.

> **Limite de sécurité**
> Le hachage est préférable au stockage en clair, mais un compte dans une base locale n'est pas une authentification centrale. Pour protéger des données partagées et réelles, il faut un serveur, des règles d'autorisation distantes et une procédure de récupération de compte.

Connexion et session

L'écran de connexion accepte e-mail ou téléphone. AuthService recherche la ligne puis vérifie le hash. Si le mot de passe est correct, l'identifiant est écrit dans app_settings sous la clé currentUserId. Au prochain démarrage, AuthProvider relit cette valeur; _AuthGate choisit l'écran à afficher.

Profil et suppression

UserProvider charge puis enregistre les informations via UserService. La photo est copiée dans le dossier privé de l'application; la préférence notificationsEnabled est enregistrée comme JSON. Cette préférence ne met pas en place une notification distante. Lors de la suppression du compte, les réclamations et historiques liés sont supprimés par cascade, et le service tente d'effacer les photos locales.

Mot de passe oublié

Le bouton affiche une information, mais l'envoi de réinitialisation n'est pas disponible en mode local. AuthService lance explicitement UnsupportedError, car un e-mail de récupération fiable demande un service en ligne.

09 · Gestion des réclamations

Création

Le formulaire collecte la référence de logement, l'objet, la catégorie, la description et des images facultatives. Les champs obligatoires sont vérifiés avant sauvegarde. Une nouvelle demande commence avec le statut ouverte et une entrée initiale dans l'historique.

Consultation selon le rôle

ComplaintService propose getForUser pour un compte et une méthode globale. ComplaintsScreen appelle la liste globale si le rôle principal est propriétaire; les autres rôles observent les réclamations du compte courant. Dans la version locale, cette différence concerne seulement les données du même appareil.

Modification et suppression

La modification change la référence, l'objet, la catégorie, la description et la liste de photos sans modifier le statut courant. La suppression cible l'identifiant de la demande et son userId. Les fichiers retirés ou supprimés sont effacés séparément de la base.

Statuts et historique

Statut	Interprétation
ouverte	La demande vient d'être créée et reste à traiter.
enCours	Le traitement de la demande a commencé.
cloturee	La demande est considérée terminée.

Quand un statut change, le service met à jour complaints.status et ajoute une ligne dans complaint_history dans la même transaction. La fiche détail peut alors afficher une chronologie, pas seulement le statut actuel.

10 · Photos et fichiers locaux

La base n'enregistre pas le contenu binaire des photos. ImagePicker fournit une image sélectionnée; l'application la copie dans son propre répertoire de documents puis enregistre son chemin. Le tableau des chemins est sérialisé en JSON dans les colonnes photo ou photos.

Copie d'une pièce jointe

```
final dir = await getApplicationDocumentsDirectory();
final folder = Directory(p.join(dir.path, 'complaint_photos'));
await folder.create(recursive: true);
final saved = await File(sourcePath).copy(
  p.join(folder.path, filename),
);
// Le chemin saved.path est ensuite enregistré en base.
```

Points à améliorer pour la production

- Limiter le poids et le nombre d'images.
- Gérer les erreurs de permission, de copie ou d'espace disque.
- Nettoyer les fichiers orphelins si le formulaire est abandonné.
- Prévoir une sauvegarde conjointe du fichier SQLite et des images référencées.
- Décider si les photos doivent être chiffrées ou supprimées automatiquement après un délai.

11 · État Flutter avec Provider

Les providers sont des ChangeNotifier. Un écran utilise context.watch ou Consumer pour écouter les changements, et context.read pour lancer une action sans s'abonner au rebuild.

État	Usage
isInitialized	Indique que la session a été lue au démarrage.
isLoading	Désactive des boutons ou montre une progression pendant une action.
error	Expose une erreur à afficher dans un SnackBar ou une zone d'état.
user / complaints	Données courantes chargées depuis les services.

Streams des réclamations

ComplaintService émet d'abord la liste de SQLite puis émet une liste actualisée lorsqu'une opération connue change les données. ComplaintProvider s'abonne au flux, remplace sa liste et appelle notifyListeners. Ce flux rafraîchit l'interface locale; il ne synchronise pas le réseau.

Erreurs

Une opération asynchrone peut échouer : contrainte unique, saisie incorrecte, base indisponible ou fichier photo absent. Les providers conservent un message d'erreur et les écrans affichent un retour. Une évolution utile consiste à différencier les types d'erreur pour proposer une action claire.

12 · Lancer le projet sur Android

Préparation

- Installer Flutter, Android Studio, l'Android SDK et Platform Tools.
- Créer puis démarrer un appareil virtuel dans Android Studio > Device Manager.
- Ouvrir le dossier qui contient pubspec.yaml, lib et android.
- Vérifier que `flutter devices` affiche l'émulateur, par exemple emulator-5554.

Commandes PowerShell

```
cd C:\Users\eyakh\studenthub
flutter pub get
flutter devices
flutter run -d emulator-5554
```

Les étapes de flutter run

Étape	Description
Résolution	Flutter vérifie les packages et prépare les sources.
assembleDebug	Gradle compile l'APK et les plugins Android.
Installation	ADB installe l'APK sur l'émulateur.
Démarrage	MainActivity charge le moteur Flutter, qui exécute lib/main.dart.
Session de développement	Le terminal reste ouvert pour les logs et le hot reload; `q` quitte proprement.

Pourquoi l'écran Hello Android reste visible ?

Tant que Gradle n'a pas terminé et que Flutter n'a pas installé son APK, l'émulateur peut rester sur l'ancienne application exemple. Une compilation terminée doit ensuite afficher les messages d'installation et lancer StudentHub. Ne démarre pas deux `flutter run` sur le même projet en même temps.

Avertissements Java

L'avertissement concernant java.lang.System::load n'est pas à lui seul une erreur. Le vrai diagnostic apparaît après `FAILURE: Build failed`, sous `What went wrong`, puis dans la première cause `Caused by`.

13 · Comprendre les erreurs de compilation rencontrées

Erreur effectivement observée

Résumé du journal fourni

```
Execution failed for task ':jni_flutter:bundleLibRuntimeToDirDebug'.
JniFlutterPlugin.class already exists;
it cannot be overwritten by an incremental SerializableChange.
```

Gradle a tenté de copier deux fois la classe JniFlutterPlugin dans son répertoire de sortie incrémental. Le chemin est sous le répertoire généré build/, dans le module Android du plugin jni_flutter. Ce message vise un état de compilation intermédiaire, pas une erreur Dart de l'écran StudentHub.

Procédure de reprise

- Attendre ou arrêter proprement tout autre build Flutter/Android Studio qui utilise le même projet.
- Depuis le dossier android, arrêter les daemons Gradle avec `./gradlew.bat --stop` si plusieurs compilations se sont chevauchées.
- Depuis la racine, exécuter `flutter clean`, puis `flutter pub get`, puis une seule compilation `flutter run -d emulator-5554`.
- Si l'erreur revient après une compilation propre, garder le nouveau journal et vérifier les versions AGP/Gradle et la version du plugin jni_flutter avant de changer les versions.

> **Lire une erreur efficacement**
> 1. FAILURE / What went wrong; 2. nom de la tâche Gradle; 3. premier `Caused by`; 4. pile d'appels complète seulement si nécessaire.

Erreur Dart ou erreur Gradle ?

- Dart : chemin `.dart`, ligne et colonne; modifier le code Flutter.
- Gradle : nom de tâche, plugin, dépendance ou fichier sous `build/`; examiner configuration et sorties générées.

14 · Sécurité, limites et sauvegarde

Atouts du stockage local

- L'application peut fonctionner hors ligne.
- SQLite est adaptée à des données structurées et des volumes modestes sur un téléphone.
- Les transactions maintiennent la cohérence de plusieurs écritures liées.

Limites importantes

- Pas de partage automatique entre appareils.
- La perte ou la désinstallation de l'application peut faire perdre la base et les photos.

- Pas de récupération de mot de passe en ligne.
- Un rôle conservé dans une base locale ne sécurise pas une API ou un serveur.
- Les photos résident dans des fichiers séparés de SQLite.

Précautions de sécurité

- Ne jamais enregistrer le mot de passe en clair ni l'ajouter aux logs.
- Garder les requêtes paramétrées et limiter la collecte de données personnelles.
- Pour un service réel, déplacer authentification, autorisations et données partagées vers un backend.
- Définir des règles de sauvegarde, de restauration et de suppression des fichiers associés.

Faire évoluer le schéma

La base actuelle est en version 1. Pour ajouter ou renommer des colonnes sans perdre les données, incrémenter la version dans openDatabase et implémenter onUpgrade. Une migration transforme la structure ou les anciennes valeurs; elle doit être vérifiée sur une copie de base contenant déjà des données.

15 · Améliorations et feuille de route

Priorité	Amélioration	Valeur
1	Migrations de base versionnées et testées.	Préserver les comptes existants pendant les changements de schéma.
2	Module locations et relations vérifiées.	Savoir à quel logement une plainte est liée et qui y a accès.
3	API et authentification serveur.	Partager les réclamations entre le locataire et le propriétaire.
4	Règles d'autorisation détaillées.	Définir qui peut éditer, supprimer ou changer un statut.
5	Tests de services et de parcours.	Vérifier comptes, doublons, relations, cycle de vie et navigation.
6	Sauvegarde et export contrôlés.	Récupérer les données et les photos en cas de changement d'appareil.
7	Adapter le stockage à d'autres plateformes.	sqflite dans la configuration actuelle ne cible pas Web/Windows/Linux.

Plan d'évolution possible

- Créer locations avec une référence stable, propriétaire, adresse et état du contrat.
- Remplacer locationId texte par une clé étrangère après migration.
- Définir l'API, les rôles, la synchronisation hors ligne et la résolution de conflits.
- Ajouter notifications, export des données et écran d'administration selon les besoins réels.

16 · Exercices et questions de soutenance

Exercice 1 · E-mail unique

Pourquoi `ETUDIANT@EXEMPLE.COM` et `etudiant@exemple.com` sont-ils traités comme le même compte ? Parce que le service normalise l'e-mail et que la colonne possède UNIQUE COLLATE NOCASE.

Exercice 2 · Historique complet

Comment éviter une réclamation sans statut initial ? En insérant réclamation et première ligne d'historique dans la même transaction SQLite.

Exercice 3 · Suppression d'un compte

Que faut-il supprimer ? Le compte, ses réclamations et leur historique par cascade. Les photos doivent être supprimées séparément, puisqu'elles sont des fichiers.

Exercice 4 · Partage entre deux téléphones

Est-ce que la synchronisation apparaît en changeant seulement le plugin SQLite ? Non. Il faut un serveur ou une API, une authentification, des permissions et une stratégie de synchronisation. SQLite peut rester le cache hors ligne du téléphone.

Exercice 5 · Nouvelle colonne priorité

Comment ajouter une colonne priority sur une base déjà installée ? Augmenter la version et écrire une migration `onUpgrade` avec ALTER TABLE et une valeur par défaut appropriée; ne pas recréer la base en effaçant les données.

Questions à préparer

- Pourquoi cette version utilise-t-elle SQLite locale ?
- Quel est le rôle d'un provider par rapport à un service ?
- Pourquoi créer une transaction pour une réclamation ?
- Où se trouvent les photos et comment la base les retrouve-t-elle ?
- Pourquoi les comptes ne se synchronisent-ils pas entre appareils ?
- Que faut-il ajouter pour réinitialiser un mot de passe par e-mail ?

17 · Glossaire et fichiers importants

Terme	Définition
SQLite	Moteur relationnel embarqué, généralement stocké dans un fichier local.
sqflite	Plugin Flutter d'accès SQLite sur certaines plateformes natives.
CRUD	Create, Read, Update, Delete : créer, lire, modifier, supprimer.
Clé primaire	Identifiant unique d'une ligne.
Clé étrangère	Lien vers une ligne d'une autre table.
Transaction	Groupe d'opérations validées ensemble ou annulées ensemble.
Migration	Passage versionné d'un schéma vers une structure plus récente.
Provider	Mécanisme Flutter de partage d'état entre widgets.
Hash et sel	Données utilisées pour vérifier un mot de passe sans conserver le texte brut.
API	Interface réseau permettant à l'application d'échanger avec un serveur.

Fichiers à ouvrir en premier

- lib/main.dart : démarrage, providers et choix de l'écran initial.
- lib/services/app_database.dart : nom, version et tables SQLite.
- lib/services/auth_service.dart : inscription, connexion et session.
- lib/services/user_service.dart : lecture et sauvegarde du profil.
- lib/services/complaint_service.dart : demandes, statut et historique.
- lib/providers/ : état chargé par les écrans.
- lib/screens/ : formulaires, listes, détail et profil.
- pubspec.yaml : dépendances et configuration Flutter.

• android/ : activité native et compilation Gradle.

Documentation technique

Documentation Flutter : docs.flutter.dev. Documentation SQLite : sqlite.org/docs.html. Références des packages : pub.dev/packages/sqflite, provider, path_provider, image_picker et cryptography. Comparer ces pages aux versions épinglées dans pubspec.lock avant une mise à jour.

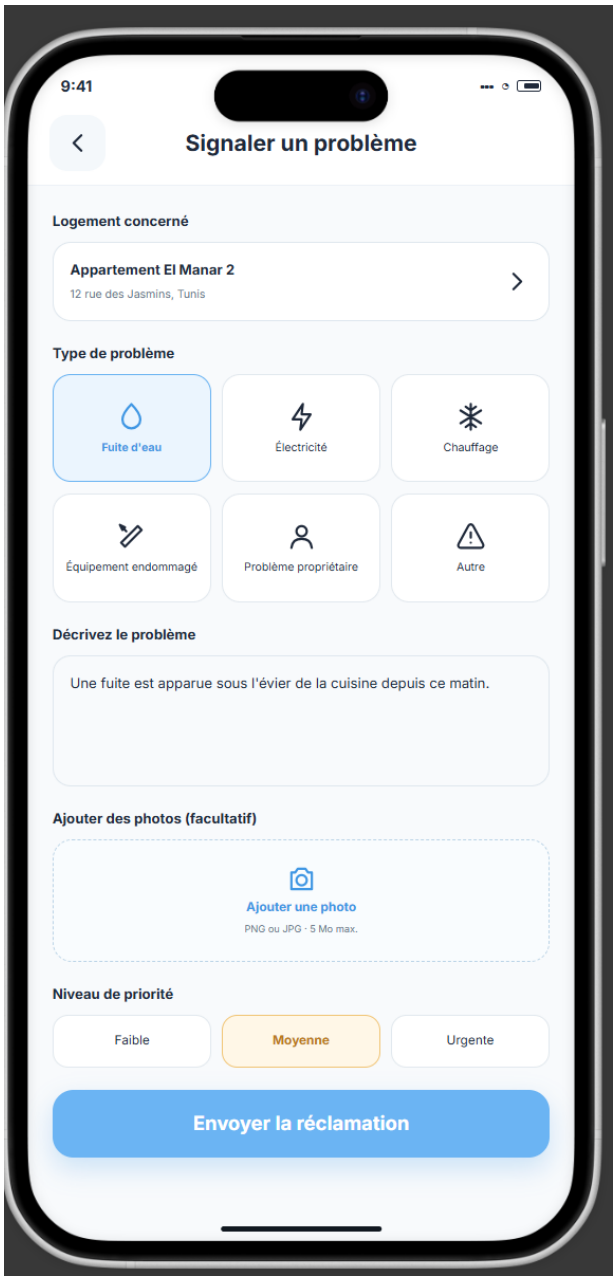
Conclusion
StudentHub est une base Flutter organisée autour de deux modules livrables : utilisateurs et réclamations. Les écrans s'appuient sur des providers, lesquels appellent des services qui enregistrent les données dans SQLite. Pour évoluer vers un vrai service partagé, il faudra ajouter un backend, un module de logements, des règles d'accès et une stratégie de synchronisation.

FIN DU GUIDE

18. Maquettes de l’application

Les cinq captures reçues définissent une direction visuelle commune : fond gris bleuté très clair, surfaces blanches, contours fins, boutons bleu clair, coins arrondis et titres bleu nuit. Les écrans ci-dessous servent de références fonctionnelles et visuelles.

Formulaire - signaler un problème



Dans Flutter

Le formulaire reprend le choix du logement, une grille de catégories, la description, l’ajout de cinq photos maximum et le niveau de priorité. Le titre de la demande reste disponible pour identifier clairement la réclamation dans la liste.

Les photos sont copiées dans le stockage local de l’application. La réclamation et sa priorité sont enregistrées dans SQLite. Une migration de base ajoute le champ de priorité aux installations déjà existantes sans supprimer leurs données.

Au premier envoi, l’application ouvre une confirmation avec une référence calculée depuis l’identifiant local. La modification d’une réclamation revient à l’écran précédent.

État livré

Écran Flutter aligné sur la maquette et relié à SQLite. Le champ logement reste une référence saisie; le catalogue de logements n’est pas encore un module du projet.

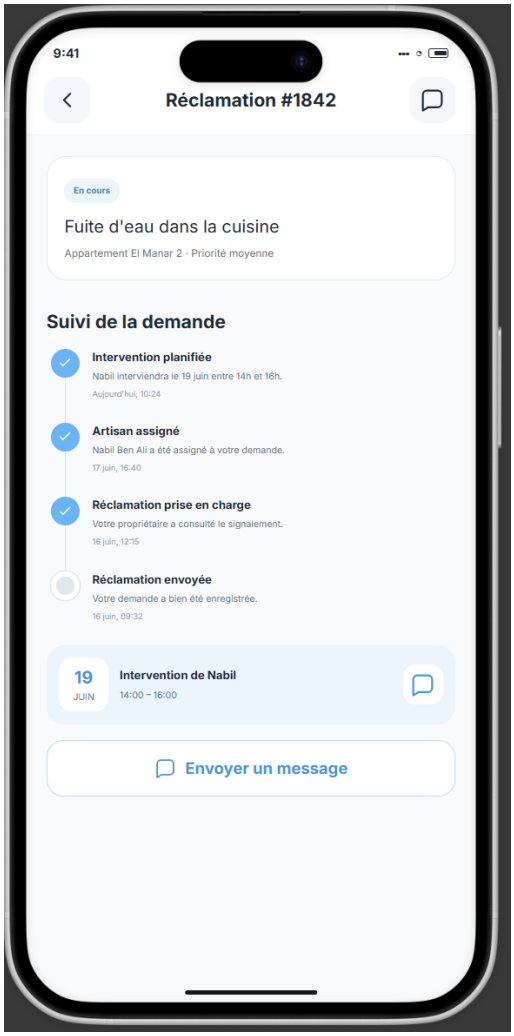
Maquette de référence : formulaire de signalement fourni dans la conversation.

Confirmation et suivi

L’envoi et le suivi forment une suite directe : l’utilisateur voit la confirmation, puis peut consulter le statut et l’historique enregistré.



Confirmation de dépôt



Détail et historique

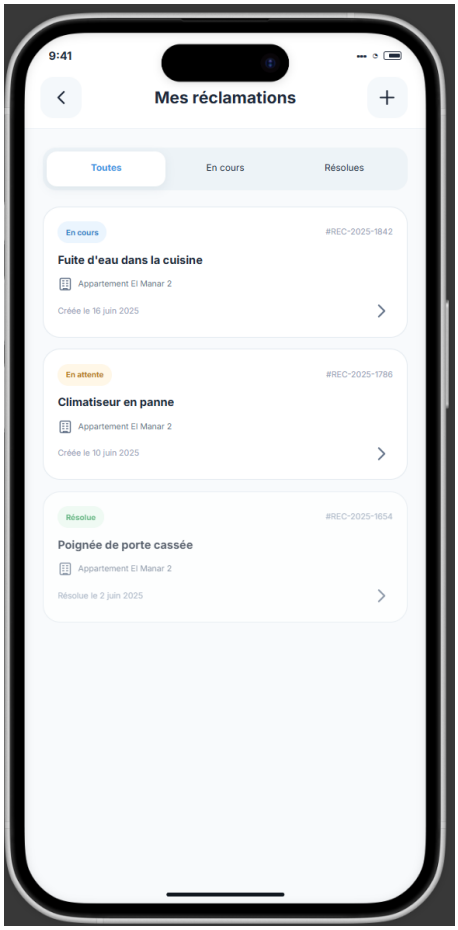
Ce que le modèle permet actuellement

La base conserve les statuts « en attente », « en cours » et « résolue » ainsi que leur date dans l’historique. Le détail affiche une chronologie à partir de ces événements et permet au propriétaire de mettre le statut à jour.

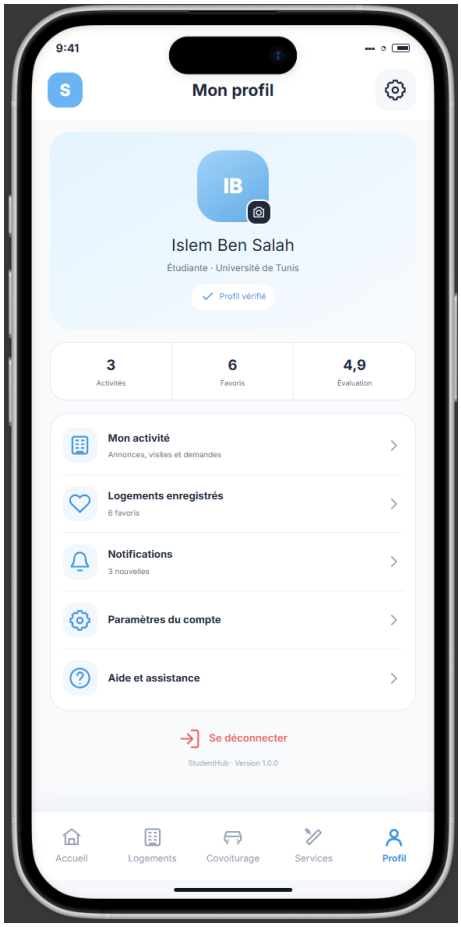
Les messages, le nom d’un artisan affecté, les créneaux d’intervention et les notifications poussées ne sont pas enregistrés par le modèle actuel. Ces éléments de la maquette demandent des tables et des écrans supplémentaires avant de pouvoir fonctionner.

Liste des réclamations et profil

La liste regroupe les demandes dans trois filtres. Le profil met en évidence des chiffres calculés depuis les réclamations locales et donne accès aux réglages qui existent déjà.



Mes réclamations



Mon profil

Différences à connaître

Les filtres et les badges de priorité/statut utilisent les données SQLite réelles. Le profil calcule les nombres de réclamations, de demandes en cours et résolues. Il garde la modification du profil, la photo locale, la préférence de notification, la déconnexion et la suppression du compte.

Le projet n'a pas de données de favoris, d'évaluation, de logements, ni de messagerie. Ces cartes de la maquette ne sont donc pas présentées comme des fonctions existantes.

Navigation

La barre de navigation actuelle expose les modules implémentés : Accueil, Réclamations et Profil. Les destinations Logements, Covoiturage et Services visibles sur la capture du profil ne sont pas encore développées.

Verification par e-mail

Inscription StudentHub, codes de confirmation et service Resend

Ce qui est implemente : l'application demande un code a six chiffres a l'inscription, le verifie, conserve l'etat dans SQLite et affiche le badge **Verifie** sur le profil. Les comptes valides attendent ensuite l'approbation de l'administrateur.

Parcours dans l'application

1 Inscription	Le compte est cree dans la base SQLite de l'appareil avec l'e-mail non verifie et le statut en attente.
2 Code e-mail	Le serveur Node envoie un code avec Resend. Le code expire apres 10 minutes et les essais sont limites.
3 Confirmation	L'application confirme le code au serveur puis marque l'adresse comme verifiee dans SQLite.
4 Approbation admin	L'administrateur valide ou bloque les comptes locaux. Seuls les comptes verifies et valides peuvent se connecter.

Architecture actuelle

Flutter appelle les routes du service Node a l'adresse definie par `ACCOUNT_API_URL`. Le serveur utilise la cle Resend, conserve les codes temporairement en memoire et appelle l'API Resend. La cle ne doit jamais etre ajoutee au code Flutter.

SQLite reste local a chaque appareil. L'ecran Gestion des comptes ne voit que les comptes presents dans la base de l'appareil administrateur. Pour une vraie administration partagee entre utilisateurs, il faudra une base serveur commune.

Configurer Resend et lancer

Guide de mise en route pour l'envoi des codes de verification

1. Compte et expediteur Resend

Creez une cle API dans Resend. Ajoutez un domaine d'envoi que vous possedez et terminez sa verification DNS, puis utilisez une adresse sur ce domaine comme expediteur. La cle API est secrete et doit rester cote serveur.

> Pour un essai, `onboarding@resend.dev` est reserve a l'envoi vers l'adresse associee au compte Resend. Pour les inscriptions d'autres personnes, configurez un domaine verifie dans Resend.

2. Variables privees du serveur

Copiez le modele `backend/.env.example` vers `backend/.env`, puis completez les valeurs. Le fichier prive est ignore par Git.

```
PORT=6500
RESEND_API_KEY=votre_cle_resend_complete
RESEND_SENDER_EMAIL=welcome@votre-domaine-verifie.com
RESEND_SENDER_NAME=StudentHub
```

Dans PowerShell, depuis la racine StudentHub, demarrez le service et laissez ce terminal ouvert :

```
node backend/welcome_mail_server.mjs
```

La sortie doit annoncer le port configure. Pour un test local depuis un deuxieme terminal PowerShell :

```
$body = @{ email = 'votre-email'; firstName = 'Eya' } | ConvertTo-Json -Compress
Invoke-RestMethod -Method Post -Uri 'http://localhost:6500/verification-email' -ContentType 'application/json' -Body $body
```

3. Connecter l'application Android

L'application Flutter exige une URL HTTPS publique pour appeler le serveur. `localhost` sur l'emulateur Android designe l'emulateur lui-meme et le service local utilise HTTP : une URL HTTPS hebergee ou un tunnel HTTPS est necessaire pour ce parcours depuis l'application.

```
flutter run -d emulator-5554 --dart-define=ACCOUNT_API_URL=https://votre-api.example.com
```

> La cle doit rester dans le fichier backend/.env, jamais dans un message, une capture d'ecran, un commit Git ou une option dart-define. Si une cle a ete partagee, revoquez-la dans Resend et remplacez-la.

En cas d'erreur HTTP, consulter le terminal Node : le serveur indique le code de refus de Resend. Un code 401 indique une cle invalide ou revoquee ; un code 422 signale une requete ou un expediteur a corriger.